Lab 06 — Data and volumes

Theory — why everything a container writes vanishes with it, the three ways to give it a disk that outlives it, and what rootless mode changes about permissions.

Objectives

- Understand why a container's *writable layer* is not a place to store anything.
- Tell a **named volume**, a **bind mount** and a **tmpfs** apart, and know which one to pick.
- Read and write `-v` and `--mount`, including `:ro`, `:U` and `:Z`.
- Explain who owns the files in a volume under rootless, and why `ls` answers "Permission denied".
- Back up and restore the data of a containerised PostgreSQL in two different ways.

---

1. The writable layer is scratch paper

In lab 02 you saw that an image is a stack of read-only layers. When a container starts, the engine adds a **writable** layer on top: empty, and private to that container. Everything the process writes — new files, modified files, "deleted" files — lands in that layer. `podman diff container` lists it.

> **→ LINUX**
> **overlayfs**, a kernel filesystem, does the stacking. It presents one merged view of several directories: the *lowerdirs* (the image layers) and one *upperdir* (the writable layer). Writing a file creates it in the upperdir. Modifying a file from a lower layer first **copies** it upward (*copy-up*), then modifies the copy. Deleting a file drops a *whiteout*, a marker that hides it. The image itself is never touched.

For data that matters, this layer has three flaws. It **dies with the container**: `podman rm` erases it, and containers get replaced more often than you think (new image, new parameter, restart by an orchestrator). It is **slow** under heavy writes, because of copy-up and the overlay indirection. And it is **invisible** to other containers: sharing is impossible.

> **REMEMBER**
> A container is **disposable** by design. If a piece of data must outlive the container, it must not live *inside* the container. That principle drives this whole lab — and, later, the split between Pods and PersistentVolumes in Kubernetes.

2. Three ways to mount a disk

The engine can **mount** a location outside the container onto a path inside it. Whatever gets written at that path bypasses the writable layer. Three kinds exist:

Kind	Where the data lives	Who manages it	When to use it
Named volume	In the engine's storage (`~/.local/share/containers/storage/volumes/`)	The engine: `podman volume …`	Data of a database, a cache, a queue: anything that must survive without you needing to know *where*
Bind mount	Any directory or file on the host, chosen by you	You	Source code during development, a configuration file, an output folder
tmpfs	In RAM, never on disk	The kernel	Temporary files, decrypted secrets, throwaway caches

Two syntaxes describe them. The short one, `-v source:destination[:options]`, is the most common; the long one, `--mount type=…,source=…,destination=…`, reads better and lets fewer mistakes through silently:

```
podman run -v pgdata:/var/lib/postgresql/data …                    # named volume
podman run -v ./html:/usr/share/nginx/html:ro …                    # bind mount, read-only
podman run --tmpfs /scratch:size=64m …                             # tmpfs
podman run --mount type=volume,source=pgdata,destination=/var/lib/postgresql/data …
podman run --mount type=bind,source=./html,destination=/usr/share/nginx/html,ro …
```

The rule that tells them apart: a source that starts with `/`, `./` or `~` is a host path, hence a **bind mount**; a plain name is a **volume**. Forget the `./` in front of `html` and you create an empty named volume called `html` — one of the most frequent mistakes.

3. Named volumes

A named volume is a directory the engine creates, names, and keeps until you delete it. `podman volume create pgdata` creates it; `podman run -v pgdata:…` mounts it (and creates it if it does not exist yet); `podman volume inspect pgdata` reveals its `Mountpoint` on the host; `podman volume rm pgdata` destroys it — and refuses while a container is using it.

On the **first** mount over a directory that is not empty in the image, the engine **copies** the image's content into the empty volume (`NeedsCopyUp: true` in the inspect output). That is what lets you mount a volume on `/etc/nginx` without ending up with an empty directory. Later mounts copy nothing.

> **PITFALL**
> Many images declare `VOLUME /var/lib/postgresql/data` in their Dockerfile. Start `postgres` **without** `-v` and the engine still creates a volume, an **anonymous** one, named by a 64-character hash. `podman rm` of the container does not remove it (`rm -v` does). A few weeks later `podman volume ls` lists dozens of orphaned volumes and nobody knows which holds what. Always name your volumes.

→ POSTGRESQL
The official image runs `initdb` at startup **if and only if** the `PGDATA` directory (`/var/lib/postgresql/data`) is empty: it then creates the cluster, the `postgres` superuser with the password from `POSTGRES_PASSWORD`, and runs the scripts in `/docker-entrypoint-initdb.d/`. If the directory already holds data, it reuses it without initialising anything — and ignores `POSTGRES_PASSWORD`. A persistent volume therefore changes how the container behaves on its second start.

4. Bind mounts

A bind mount makes a host path visible inside the container, **with no copy**: it is the same directory, seen from two places. Edit `html/index.html` on the host and nginx serves the new version at once; write from the container and a file appears in your folder. It is the tool for development (reload without a rebuild) and for configuration (`-v ./nginx.conf:/etc/nginx/nginx.conf:ro`).

The `:ro` option forbids writes from the container: an attempt fails with `Read-only file system`. Make it your default, and drop it only when the container really has to write.

→ SECURITY
On Red Hat, Fedora and CentOS, **SELinux** labels every file and forbids a container from reading anything that does not carry the `container_file_t` label. A bind mount fails there with `Permission denied`, even as root, until you add `:z` (shared between containers) or `:Z` (private to this container), which relabel the directory. Ubuntu (hence WSL) uses AppArmor, not SELinux: `:Z` is accepted there and does nothing. Get into the habit of writing it anyway: your commands will then work on the company's RHEL servers too.

→ WINDOWS / WSL
Your Windows files are visible under `/mnt/c/…`, but that mount (`drvfs`, 9p protocol) is **slow** and does not handle Linux permissions: a bind mount from `/mnt/c` works, then surprises you with `chmod` calls that do nothing and long access times. Keep the lab files in the Linux filesystem (`~/labo-docker`) and open them from Windows through `\\wsl$\Ubuntu\home\…` when you need to.

5. Rootless: who owns the files?

In lab 01 you saw that the `root` of a rootless container is your own user. The rest of the table follows: UID 1 in the container maps to the first UID of your `/etc/subuid` range, UID 2 to the second, and so on.

→ LINUX
A **user namespace** gives a process its own UID table. The kernel translates every container UID into a real host UID according to `/proc/<pid>/uid_map`, three columns: *container start*, *host start*, *length*. On a standard Ubuntu/WSL, `podman unshare cat /proc/self/uid_map` prints `0 1000 1` (your UID becomes root) and then `1 100000 65536` (the rest of the table points into the subuid range). The image's `postgres` user, UID 70, therefore writes its files as host UID **100069**.

The very concrete consequence: seen from the host, the files in the PostgreSQL volume belong to UID 100069, and your `ls` on the `Mountpoint` answers `Permission denied`. That is not a failure; it is isolation doing its job. To get in, enter the namespace: `podman unshare ls -ln …` then shows `70 70`, exactly as inside the container.

The reverse problem hits bind mounts: your folder belongs to UID 1000, which the container sees as **root**. A process running as `--user 1000` inside the container (host UID 100999) has **no rights**

at all on it. Three remedies, from the simplest to the cleanest:

Remedy	Effect	When
`:U` on the mount	The engine runs a recursive `chown` to the container's UID	Folder dedicated to that container
`--userns=keep-id`	Your UID keeps the same number inside the container	Development tools that write into your sources
`podman unshare chown`	You change the owner "as the container sees it"	Repair after the fact

> **PODMAN**
> Docker in its classic mode has no such problem: the daemon is root, and a file written by UID 70 in the container *really* belongs to UID 70 on the host — which is a different problem, a security one. `podman unshare`, `:U` and `--userns=keep-id` are specific to Podman. So are `podman volume export` and `podman volume import`, which archive a volume to a tar file without any container; Docker has no equivalent.

6. tmpfs: a disk in memory

`--tmpfs /scratch:size=64m` mounts a filesystem in RAM: fast, bounded in size, wiped when the container stops, never written to the host's disk. It is the place for a large temporary file, or for a decrypted secret that must leave no trace. Watch the meter, though: the memory a tmpfs uses counts against the container's `--memory` limit (lab 10).

7. Backing up and restoring

Two families, and neither replaces the other:

- **Logical backup**: you ask the software to export its data in its own format. For PostgreSQL, `podman exec db pg_dump -U postgres > dump.sql` produces replayable SQL that is **consistent** even while the database runs, and portable across versions. This is the reference backup.
- **Physical backup**: you archive the volume's files. `podman volume export pgdata -o pgdata.tar` (Podman), or the portable method `podman run --rm -v pgdata:/data:ro -v ./:/backup alpine tar czf /backup/pgdata.tgz -C /data .`. Fast and exact, but **the container must be stopped**: copying the files of a database while it writes yields an inconsistent archive, sometimes beyond repair.

8. In the workplace

- **The database** lives in a named volume on the developer's machine and in CI; in production it most often sits **outside the container** (managed service) or on network storage the orchestrator attaches to the node (PersistentVolume, lab 15). Never in a bind mount on an application server's disk.
- **Configuration** (`application.yaml`, `nginx.conf`, certificates) arrives through a read-only bind mount — or, in lab 08, through environment variables and secrets.
- **Logs** do not go to a file inside the container: they go to standard output, and the engine collects them (lab 10). A `logs/` folder mounted as a volume is a sign of an application that was not adapted properly.

- **User uploads** (photos, PDFs) go to object storage (S3, MinIO), not to a local volume: a second replica would never see them.

Remember

- The writable layer dies with the container; data that must survive does not live *inside* the container.
- Named volume for the engine's data, bind mount for your files, tmpfs for memory.
- A source starting with `/`, `./` or `~` is a bind mount; a plain name is a volume — forgotten `./` = empty volume.
- `VOLUME` in an image creates anonymous volumes: name yours, and delete with `rm -v`.
- `:ro` by default; `:Z` for SELinux; `:U` or `--userns=keep-id` when the container's UID does not own the folder.
- Under rootless, container UID 70 is host UID 100069: use `podman unshare` to see the files.
- `pg_dump` for a consistent backup of a running database; `volume export` or `tar` only with the container stopped.

Vocabulary

writable layer: overlay layer private to the container, destroyed with it. — **named volume**: engine-managed, persistent directory. — **anonymous volume**: volume created by `VOLUME` with no name. — **bind mount**: host directory mounted into the container. — **tmpfs**: in-memory filesystem. — **copy-up**: copy of the image's content into an empty volume on first mount. — **user namespace / subuid**: UID translation table under rootless. — **`podman unshare`**: run a command inside Podman's user namespace. — **logical / physical backup**: export by the software / copy of the files.